

Project Executable Files

Date	2 june 2026
Team ID	xxxxxx
Project Name	Opti crop- smart agricultural
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

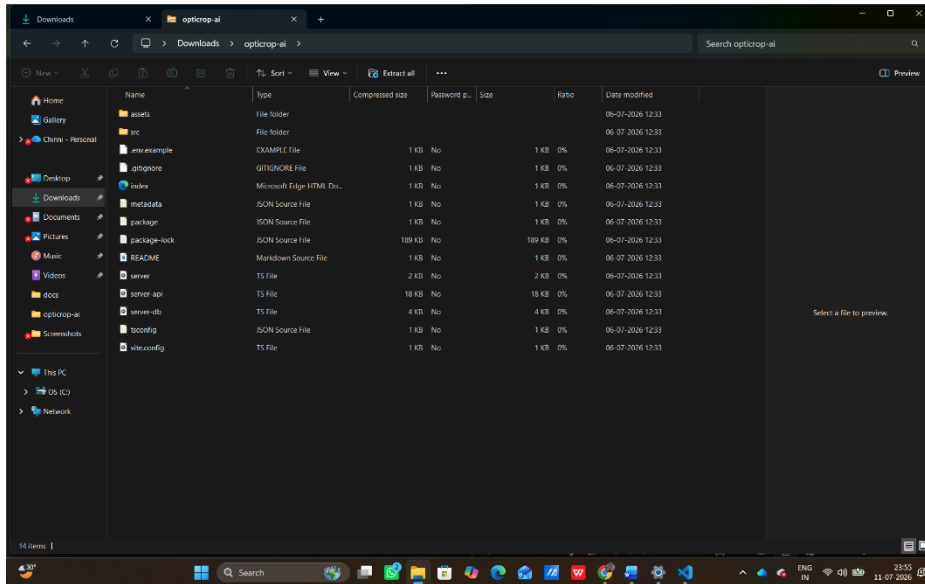
Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	yes

10	Demo video or walkthrough (if required)	yes
----	---	-----

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://opticrop-ai-463011982132.asia-southeast1.run.app/
Login Credentials (Demo)	[Provide demo username and password for evaluator access]
Platform / Hosting Provider	[e.g., Render, Vercel, AWS, Heroku, Azure]
Repository Link	https://github.com/Sumanthkalyan-07/opticrop-ai
Demo Video Link	

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites

Story Duration: 1h 0m

To Do

Assigned to Sumanth Kurakula

Description

The project was developed using a set of powerful Python-based tools and libraries for data processing, machine learning, visualization, and deployment. These technologies provide the foundation for efficient model development, analysis, and application deployment.

Anaconda Navigator: A desktop graphical user interface used to launch applications and manage Python packages and environments for data science. <https://www.anaconda.com/download>

PyCharm: An Integrated Development Environment (IDE) specifically designed for Python programming, offering code analysis and debugging tools. <https://www.jetbrains.com/pycharm/>

NumPy: Core library for numerical computing in Python, providing support for large, multi-dimensional arrays and mathematical functions. <https://numpy.org/doc/stable/>

Pandas: Data manipulation and analysis library offering data structures and operations for manipulating numerical tables and time series. <https://pandas.pydata.org/docs/>

Scikit-learn: Machine learning library featuring classification, regression, and clustering algorithms like support vector machines and random forests. <https://scikit-learn.org/stable/>

Matplotlib: Comprehensive library for creating static, animated, and interactive visualizations in Python. <https://matplotlib.org/stable/>

Seaborn: Data visualization library based on matplotlib that provides a high-level interface for drawing attractive statistical graphics. <https://seaborn.pydata.org/>

Flask: A lightweight WSGI web application framework used to build web APIs and deploy machine learning models. <https://flask.palletsprojects.com/>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Out-of-range Input Errors: If a user inputs highly unrealistic or negative soil values (e.g., negative NPK or pH values), the model might output an inaccurate crop recommendation instead of an error.	Status: Form validation is active on the HTML frontend to restrict inputs to positive numbers, but backend range handling will be optimized in the next iteration.
2	Limited Climate Realism: The system relies on static manual inputs for temperature and rainfall rather than pulling real-time, live regional weather data from an external API.	Workaround: Users must look up and manually enter local weather averages from a weather service application.
3	Data Coverage Constraint: The system is only trained on a specific subset of standard agricultural crops (e.g., rice, maize, cotton). It cannot recommend local wild or niche regional crops.	Status: A disclaimer has been added to the user interface to specify the exact list of crops the system can accurately predict.